

ICS-PA1 实验报告

241220029 杨智宇

一、思考题

(一). PA1-1 思考题

1. C 语言中 struct 和 union 的区别

struct 和 union 都是用来把一组数据类型（可以不同，也可以相同）构造成为一个整体的数据类型。区别是两者在内存分配的时候，struct 会为其中的每个成员开辟空间，而 union 中的所有数据共享同一段内存空间，其大小由 union 中内存大小最大的元素决定。

举个例子：以下测试代码：

```
1  #include<stdio>
2  int main(){
3      struct A{
4          int a;
5          double b;
6          char c;
7          int d;
8      };
9      union B{
10         int a;
11         double b;
12         char c;
13         int d;
14     };
15     A jiegouti;
16     B lianheti;
17     printf("结构体的空间大小是： %d\n",sizeof(jiegouti));
18     printf("联合体的空间大小是： %d\n",sizeof(lianheti));
19     return 0;
20 }
```

当我要求编译器输出结构体和联合体的内存大小时，结果会是：

```
结构体的空间大小是： 24
联合体的空间大小是： 8
```

这是因为两者在内存分配上的逻辑不同。这导致了在任何同一时刻，联合体只存放了一个被先选中的成员，而结构体的所有成员都存在。

2. 寄存器结构体的参考实现为什么把部分 struct 改成了 union

寄存器的本质是一块确定的内存空间，用来暂时存放一些数据。如果用 struct 的话，会导致这个空间同时存在了好几种不同的数据，就会导致错误，而 union 的确定空间、只能同时存在一个成员数据类型的特性正好就是我们需要的。

下面我来简要分析一下寄存器这里的 union 实现了什么。

```

// general purpose registers
union
{
    union
    {
        union
        {
            uint32_t _32;
            uint16_t _16;
            uint8_t _8[2];
        };
        uint32_t val;
    } gpr[8];
    struct
    { // do not change the order of the registers
        uint32_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
    };
};

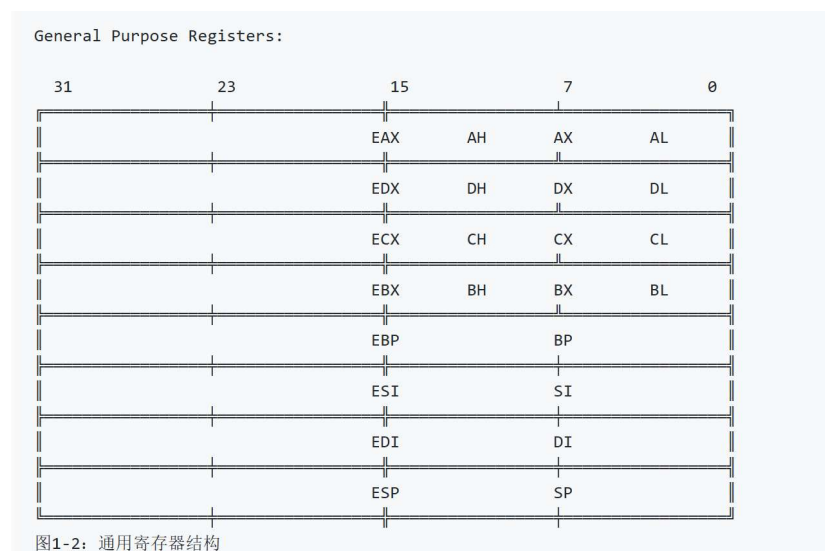
```

这里最内层 union 表示 _32 、 _16 、 _8[2] 共用一块内存空间；

中间层 union 表示内层 union 和 uint32_t val 共用一块内存空间，共同构成一个 gpr，这里声明了一个 gpr 数组，由 8 个 gpr 构成；

最外层 union 表示上述的 8 个次内层 union (gpr) 和 uint32_t eax, ecx, edx, ebx, esp, ebp, esi, edi 寄存器共用一块内存；

这样对于寄存器:eax, ecx, edx, ebx, esp, ebp, esi, edi 赋值的时候，就相当于在 gpr[i] 上给赋值了，因为他们在同一个确定的内存空间中，如下图：



(二). PA1-3 思考题

1. 正（负）上溢

假设 $fa.exponent = fb.exponent = 0xfe$ ，且 fa 和 fb 均为最大正规格数（尾数全 1），这样 $fa*fb$ 和 $fa+fb$ 的结果阶数都会满足 $exp \geq 0xff$ ，导致上溢，结果为正无穷（负上溢同理， fa 和 fb 均改为负的即可）

2. 正（负）下溢

假设 f_a, f_b 的阶码都是 00000001, $f_a = +1.000000000000000000000001 * 2^{-126}$;
 $f_b = -1.0000000000000000000000010 * 2^{-126}$, 这样 $f_a * f_b$ 的阶码直接小于 -127, 下溢;
而 $f_a + f_b$ 结果为 $-0.000000000000000000000001 * 2^{-126}$, 规格化后发生下溢。

（正负同理，将 f_a, f_b 的符号位同时取反即可）

二、心得感想

1. 一上来做 PA 的感觉有点懵，就是知道自己在做什么，但是代码的上下文看不太明白，一些变量的用处不是很清楚，我感觉这是因为缺少对整体代码框架的了解。不过在 PA1-3 的时候，这个现象有所缓解。
2. 我对强制类型转换、手动写位扩展的区别、以及一些二进制数在 C 中的操作不是很明白（这在 PA1-2 中导致了一些麻烦），同样的，在 PA1-3 这种有总览、上下文有参考的任务时，有所缓解。
3. （小建议）我感觉可以在教程中多加一些对每个代码片段引用的头文件（以及各种在整个 PA 中重要的变量）的简要介绍，这样可以让整体实现的时候思路更清楚一些，不至于完全盲目的写。